

VULNSCAN LITE

Passive Web Security Scanner

Internship Project Report

Project	VulnScan Lite — On-Demand Passive Security Health Scanner
Internship	Persevex Cyber Security Internship • Project 2 • 2026
Scope	Authorized public HTTP(S) targets; passive configuration analysis only.

Disclaimer: Only scan websites you own or have permission to assess. This tool performs passive analysis only.

1. Executive Summary

VulnScan Lite is a focused web security-health scanner designed to turn a small set of repeatable passive checks into an authenticated, asynchronous workflow. A user submits an authorized public HTTP(S) target, the API validates and queues the assessment, a Celery worker performs the scan, and the application persists the results for structured review and PDF export.

The scanner deliberately avoids exploit payloads, brute force, port scanning, directory enumeration, and authentication bypass. Its core checks cover HTTP security headers, verified TLS configuration, and conservative CMS fingerprinting. Results are converted into a deterministic 0–100 score with an A–F grade and paired remediation guidance.

The project separates the React/Vite dashboard, FastAPI API, PostgreSQL persistence, Redis task queue, Celery worker, and independent scanner package. This separation keeps the scanner logic testable and makes the system easier to extend.

2. Objectives

- Provide a simple authenticated interface for authorized passive website assessments.
- Analyze baseline HTTP security headers including CSP, X-Frame-Options, and HSTS.
- Inspect TLS using Python's verified SSL context, including protocol, certificate and cipher properties.
- Detect selected CMS families using conservative public evidence.
- Run scans asynchronously through Redis and Celery rather than blocking the API request.
- Produce deterministic scoring, remediation guidance, scan history, and PDF reports.
- Reduce misuse risk through target validation, SSRF-reduction controls, timeouts, response limits, rate limits, and ownership checks.

3. System Architecture

The application follows a separated full-stack architecture:

Layer	Technology / Responsibility
Frontend	React 19 + Vite dashboard for authentication, scan submission, progress polling, results, history, and reports.
API	Python + FastAPI + Pydantic + SQLAlchemy for authentication, scan lifecycle, history and report access.
Queue	Redis-compatible queue used by Celery to execute scan jobs asynchronously.
Worker	Celery worker executes the passive scanner and persists progress/results.
Scanner	Independent Python scanner package containing headers, TLS, CMS, safety and scoring logic.
Database	PostgreSQL stores users, scans, results and history.
Reporting	ReportLab generates PDF security-health reports with wrapped evidence and remediation text.

The scanner package is independent of FastAPI, database models and Celery. This separation allows the core checks to be unit-tested without coupling them to the web layer.

4. Scanner Methodology

4.1 HTTP Security Headers

- Required baseline: Content-Security-Policy, X-Frame-Options, and Strict-Transport-Security on HTTPS targets.
- Bonus hardening: X-Content-Type-Options, Referrer-Policy, and Permissions-Policy.
- A missing baseline header applies a 10-point penalty.

4.2 TLS Inspection

- Uses Python's verified SSL context.
- Checks certificate chain and hostname validation, TLS 1.2+ baseline, effective cipher strength of at least 128 bits, expiry, issuer and subject.
- The scanner does not retry with certificate verification disabled.

4.3 CMS Fingerprinting

- Uses conservative evidence such as generator meta tags, exact X-Powered-By values and stable public asset/path markers.
- Supported families: WordPress, Drupal and Joomla.
- Visible page text alone is not treated as CMS evidence.

5. Asynchronous Scan Workflow

The API does not perform the target scan inside the request handler. The workflow is:

- POST /api/scan validates the target and creates a scan record.
- The scan task is placed on the Redis queue.
- A Celery worker executes the scanner.
- Progress is persisted through real stages: 5% → 20% → 40% → 60% → 80% → 95% → 100%.
- The frontend polls GET /api/scan/{id}/status at approximately two-second intervals.
- Completed results can be viewed in the dashboard and exported as a PDF.

6. Scoring and Reporting

The scoring model starts at 100 points and subtracts documented penalties, then clamps the result to the 0–100 range.

Grade	Range
A	90–100
B	80–89
C	70–79
D	60–69
F	0–59

Each failed check is paired with practical remediation guidance. The final report presents the target, scan metadata, HTTP/TLS information, findings, evidence, recommendations, and CMS observations.

7. Security and Abuse Controls

- Only absolute HTTP(S) URLs are accepted; embedded credentials are rejected.
- Localhost, internal names, and non-global DNS answers are rejected; redirects are revalidated.
- Maximum of 3 redirects, 12-second timeout, 1 MB response cap, and trust_env=False.
- Rate limits: 5 scan requests per minute per IP, 10 scans per hour per user, and 3 active scans system-wide.
- Ownership checks protect scan status, reports, PDFs and history.
- Secrets are supplied through environment configuration; production rejects the development JWT secret.

Deployment security note: application-level checks reduce SSRF risk but do not eliminate DNS rebinding. Production workers should also use appropriate egress firewall controls.

8. Main API Endpoints

Endpoint	Purpose
POST /api/auth/register	Create an account.
POST /api/auth/login	Authenticate and receive a bearer token.
POST /api/scan	Validate and queue a passive scan.
GET /api/scan/{id}/status	Poll scan progress.
GET /api/scan/{id}	Retrieve completed scan details.
GET /api/scan/{id}/pdf	Download the generated PDF report.
GET /api/history	View authenticated scan history.
GET /health	Service health check.

9. Testing and Validation

The project build was validated through automated backend tests, frontend production build checks, dependency auditing, and a complete local full-stack workflow.

Validation	Result
Backend automated tests	35 / 35 passed
Frontend npm audit	0 vulnerabilities reported
Frontend production build	Passed
Full-stack local flow	Verified with FastAPI, PostgreSQL, Redis/Memurai, Celery, frontend, authentication, authorized target scanning, persistence, history and PDF generation.

The test suite covers headers, scoring boundaries, CMS evidence, SSRF validation, redirect checks, TLS decisions, PDF generation, ownership protection and scan API behavior.

10. Example Scan Result

During local functional verification, an authorized Blogger target was scanned successfully. The resulting report demonstrated the complete scan lifecycle and PDF generation.

Metric	Observed result
Score / Grade	70 / 100 • Grade C
HTTP status	200
TLS	TLSv1.3 • TLS_AES_256_GCM_SHA384
Certificate	Valid during the scan; Google Trust Services WR2 issuer
CMS	No supported CMS detected; low confidence
Findings	Missing CSP, X-Frame-Options and HSTS; additional policy warnings were reported.

This example is an assessment of configuration posture, not a penetration-test result. The scanner intentionally remains within its passive-analysis boundary.

11. Limitations and Future Improvements

Current limitations

- Three supported CMS families.
- TLS inspection is based on the negotiated handshake.
- Rate limiting is in-process.
- Database schema creation currently occurs at startup.
- Bearer token storage uses localStorage.
- There is no browser-test harness in the project.

Future engineering steps

- Introduce managed database migrations.
- Move rate limiting to distributed storage.
- Add a distributed scan-capacity lock.
- Expand CSP parsing and policy analysis.
- Use a stronger session model.
- Add CI/CD dependency and container scanning.

12. Deployment and Repository Structure

The project includes deployment configuration for a separated frontend/backend/worker architecture. The repository contains the application source, documentation, tests, environment template, Docker configuration and Render configuration.

Important repository hygiene: local .env files, node_modules, Python caches, local databases and build artifacts should not be committed. The included .env.example documents the required configuration without exposing secrets.

13. Conclusion

VulnScan Lite demonstrates a focused approach to passive web security assessment. It combines a maintainable full-stack architecture with asynchronous job execution, conservative scanner logic, abuse controls, deterministic scoring, remediation guidance, persistent history and PDF reporting.

The project deliberately documents its boundaries instead of presenting a limited passive scanner as a full penetration-testing platform. This makes the system easier to understand, test, operate and extend.

Key takeaway: VulnScan Lite turns a small set of passive web checks into a repeatable, authenticated and reportable security-health workflow.

Disclaimer: Only scan websites you own or have permission to assess. Passive analysis only.